

Comprehensive Architectural Analysis of a Client-Server Messenger Application

System Architect

April 29, 2026

Contents

1	Introduction	3
2	Module Analysis	3
2.1	Common Module (<code>com.messenger.common</code>)	3
2.2	Client Module (<code>com.messenger.client</code>)	3
2.3	Client UI Module (<code>com.messenger.client.ui</code>)	4
2.4	Client Media Module (<code>com.messenger.client.media</code>)	4
2.5	Server Module (<code>com.messenger.server</code>)	4
3	System Workflows	5
3.1	Authentication & Registration Flow	5
3.2	Messaging Flow	5
3.3	Peer-to-Peer Media Signaling & Streaming Flow	5
4	Conclusion	6

1 Introduction

The Messenger Application is a robust, multi-threaded, client-server system developed in Java. It provides real-time text messaging, presence tracking, and peer-to-peer multimedia communication, including both audio and video calling functionalities. The architecture is distinctly separated into client and server components. The client side utilizes JavaFX for a responsive graphical user interface and leverages UDP alongside specialized libraries (JavaCV for video, JavaSound for audio) to facilitate low-latency media streaming. The server acts as a centralized routing hub utilizing standard Java Sockets (TCP) to manage client connections, state, and message delivery, persistently storing application data within a PostgreSQL database via JDBC.

2 Module Analysis

2.1 Common Module (`com.messenger.common`)

The `common` package acts as the foundational data layer shared between the client and server.

- `User.java` & `Message.java`: These are standard POJOs implementing `Serializable`. They represent the core entities of the domain model. The `Message` class includes an `isDelivered` boolean flag, crucial for managing offline message queues.
- `CallType.java`: An enumeration defining `AUDIO` and `VIDEO` call types.
- `NetworkPayload.java`: This class implements the Wrapper Pattern. Every piece of data sent over the TCP network is encapsulated within a `NetworkPayload` object. It contains a `PayloadType` enum (e.g., `LOGIN_REQUEST`, `SEND_MESSAGE`, `CALL_REQUEST`), an `Object` payload for data, and a `status` string. This standardization greatly simplifies the parsing logic on both ends of the socket.

2.2 Client Module (`com.messenger.client`)

This module forms the backbone of the client application logic, effectively decoupling the network operations from the UI.

- `ClientLauncher.java` & `ClientMain.java`: Entry points for the JavaFX application. They manage the primary stage and navigation between the login and chat views.
- `NetworkClient.java`: Manages the TCP socket connection to the server. It utilizes `ObjectOutputStream` and `ObjectInputStream` to transmit `NetworkPayload` instances. It spawns a dedicated daemon thread that continuously listens for incoming payloads.
- `ChatManager.java`: The core orchestrator on the client side. It abstracts network operations for the UI. It heavily utilizes callback interfaces (`MessageListener`, `CallListener`, etc.) to propagate events back to the UI. It uses `Platform.runLater()` to ensure all UI updates triggered by network events are safely executed on the JavaFX Application Thread.

2.3 Client UI Module (`com.messenger.client.ui`)

Handles all graphical interactions using JavaFX.

- `LoginView.java`: Provides a modern interface for user authentication and registration.
- `ChatView.java`: The main messaging interface. It maintains the user list, chat history, and media call controls. When a media call is initiated or received, this class dynamically provisions PiP (Picture-in-Picture) video layouts and visual audio equalizers, tying UI elements directly to the underlying media stream handlers.

2.4 Client Media Module (`com.messenger.client.media`)

Responsible for real-time peer-to-peer data transmission using UDP to minimize latency.

- `StreamSender.java` & `StreamReceiver.java`: Wrappers around `DatagramSocket`. They handle the transmission and reception of byte arrays to and from specific IP addresses and ports. The receiver calculates audio amplitude for UI visualization using root-mean-square (RMS) computations on the raw byte stream.
- `MediaCapture.java`: Orchestrates hardware capture. It uses JavaSound (`TargetDataLine`) to capture microphone input and JavaCV (`OpenCVFrameGrabber`) to capture webcam frames. Video frames are converted to JPEGs and compressed before transmission. Multi-threading is heavily used to separate audio and video capture loops.

2.5 Server Module (`com.messenger.server`)

The centralized routing and state management component.

- `MessagingServer.java`: The main server daemon. It listens on a specific port (1234) and spawns a new `ClientHandler` thread for every incoming socket connection. It maintains a `ConcurrentHashMap` of active clients for O(1) routing lookups.
- `ClientHandler.java`: Represents an active connection. It continuously reads `NetworkPayload` objects from the stream and routes them. It handles crucial signaling logic: when a `CALL_REQUEST` is processed, it dynamically injects the caller's IP address into the payload before routing it to the callee, enabling the subsequent UDP peer-to-peer connection.
- `DatabaseManager.java`: Manages JDBC connections to PostgreSQL.
- `UserDAO.java` & `MessageDAO.java`: Implement the Data Access Object pattern. They encapsulate all SQL queries. `MessageDAO` specifically handles the `is_delivered` logic; if a message is sent to an offline user, it is stored with `is_delivered = 0`. When the user logs in, the `MessagingServer` fetches these offline messages, delivers them, and subsequently marks them as delivered.

3 System Workflows

3.1 Authentication & Registration Flow

1. The user inputs credentials into the `LoginView`.
2. `ChatManager` instructs `NetworkClient` to send a `LOGIN_REQUEST` payload.
3. `MessagingServer` receives the request via the `ClientHandler`.
4. `ClientHandler` delegates to `UserDAO`, which queries the PostgreSQL database.
5. Upon success, `ClientHandler` registers the client in the `activeClients` map, broadcasts a `STATUS_UPDATE` to notify contacts that the user is online, and flushes any pending offline messages.
6. A `LOGIN_SUCCESS` payload is returned to the client, triggering UI navigation via `Platform.runLater`.

3.2 Messaging Flow

1. The sender types a message in `ChatView` and clicks send.
2. `ChatManager` creates a `Message` object and wraps it in a `SEND_MESSAGE` payload.
3. `ClientHandler` receives the payload and passes it to `MessagingServer.routeMessage()`.
4. The server checks the `activeClients` map. If the receiver is online, the message is sent through the receiver's socket, and stored in the database with `is_delivered=1`.
5. If the receiver is offline, the server skips socket transmission and directly stores the message via `MessageDAO` with `is_delivered=0`.

3.3 Peer-to-Peer Media Signaling & Streaming Flow

1. **Signaling (TCP):** The caller clicks the video call button. A `CALL_REQUEST` payload is sent to the server.
2. The server's `ClientHandler` intercepts the request, appends the caller's public/network IP address to the payload, and routes it to the callee.
3. The callee receives the request, prompts the user, and sends a `CALL_ACCEPT` payload back to the server.
4. The server appends the callee's IP address and routes the acceptance back to the caller. Both parties now possess each other's IP addresses.
5. **Streaming (UDP):** Both clients locally instantiate `StreamSender` and `StreamReceiver` instances bound to predetermined ports (e.g., 5001 for audio, 5003 for video).
6. `MediaCapture` begins pulling frames from the webcam and audio from the microphone, transmitting raw UDP packets directly between the two client IPs, completely bypassing the central server to ensure minimal latency.

4 Conclusion

The Messenger Application demonstrates a highly effective hybrid architecture. By utilizing TCP for reliable signaling, state management, and text messaging, while offloading high-bandwidth, latency-sensitive media transmission to P2P UDP streams, the system achieves both reliability and performance. The strict separation of concerns, evidenced by the distinct UI, Network, and Media packages, alongside the robust DAO layer for PostgreSQL integration, results in a highly maintainable and scalable codebase.